



Материали за подготовка - Обектно ориентирано програмиране

Клас: 12б | Предмет: Обектно ориентирано програмиране

Дата на контролната работа: _____

1. Основни принципи на ООП

1.1. Инкапсулация

Инкапсулацията е скриване на детайлите за имплементация и показване само на необходимия интерфейс.

- Скриване на вътрешната реализация от външния свят
- Контролиран достъп до данните чрез модификатори за достъп
- Използване на свойства (properties) за контролиран достъп

```
class Student {  
    private string name; // Скрито поле  
  
    public string Name { // Публично свойство  
        get { return name; }  
        set { name = value; }  
    }  
}
```

1.2. Наследяване

Наследяването е създаване на нови класове, които наследяват характеристики на съществуващи класове.

- Преизползване на код от базовия клас
- IS-A връзка (производният клас Е специален случай на базовия)
- C# поддържа само единично наследяване

```
class Animal {  
    protected string name;  
    public Animal(string name) {  
        this.name = name;  
    }  
}  
  
class Dog : Animal { // Dog наследява Animal  
    public Dog(string name) : base(name) {  
    }  
}
```

1.3. Полиморфизъм

Полиморфизмът е възможност за използване на обекти с различни типове чрез един и същ интерфейс.

- Различни класове имплементират един и същ интерфейс по различен начин
- Постига се чрез virtual и override методи
- Runtime полиморфизъм - избор на метода се прави по време на изпълнение

```
class Animal {  
    public virtual void MakeSound() {  
        Console.WriteLine("Животното издава звук");  
    }  
}  
  
class Dog : Animal {
```

```
public override void MakeSound() {  
    Console.WriteLine("Кучето лае");  
}  
}
```

1.4. Абстракция

Абстракцията е създаване на упростена версия на сложна система, която показва само важните детайли.

- Скриване на ненужните детайли
- Фокусиране върху важните аспекти
- Постига се чрез абстрактни класове и интерфейси

2. Компоненти на клас

2.1. Полета (Fields)

Полетата са променливи, които съхраняват данни в класа.

```
class Student {  
    private string name;    // Private поле  
    protected int age;     // Protected поле  
    public string school;   // Public поле  
}
```

2.2. Свойства (Properties)

Свойствата предоставят контролиран достъп до private полета.

```
class Student {  
    private string name;  
  
    public string Name {  
        get { return name; }  
        set {  
            if (!string.IsNullOrEmpty(value))  
                name = value;  
        }  
    }  
}
```

2.3. Методи

Методите са функции, които извършват действия.

```
class Calculator {  
    public int Add(int a, int b) {  
        return a + b;  
    }  
}
```

2.4. Конструктори

Конструкторът е специален метод, който инициализира обект при създаването му.

```
class Student {  
    private string name;  
    private int age;  
  
    public Student(string name, int age) {  
        this.name = name;  
    }  
}
```

```
        this.age = age;
    }
}
```

Използване на `base` при наследяване:

Когато производен клас наследява базов клас, можем да използваме `base` за извикване на конструктора на базовия клас.

```
class Animal {
    protected string name;
    public Animal(string name) {
        this.name = name;
    }
}

class Dog : Animal {
    public Dog(string name) : base(name) {
        // base(name) извиква конструктора на Animal
    }
}
```

3. Модификатори за достъп

3.1. Private

Достъпен само в рамките на същия клас.

```
class Student {  
    private string name; // Достъпен само в Student  
}
```

3.2. Protected

Достъпен в същия клас и в производните класове (наследниците).

```
class Animal {  
    protected string name; // Достъпен в Animal и наследниците  
}  
  
class Dog : Animal {  
    public void Display() {  
        Console.WriteLine(name); // Може да достъпи name  
    }  
}
```

3.3. Public

Достъпен навсякъде - в същия клас, наследниците и извън класа.

```
class Student {  
    public string name; // Достъпен навсякъде  
}  
  
// Извън класа:  
Student s = new Student();  
s.name = "Иван"; // Може да достъпи name
```

Йерархия на модификаторите (от най-ограничен към най-отворен):

1. **private** - най-ограничен (само в рамките на класа)
2. **protected** - за наследниците
3. **public** - достъпен навсякъде

4. Статични членове

4.1. Статично поле

Статичното поле е споделено между всички обекти от класа.

```
class Counter {  
    public static int count = 0;  // Статично поле  
  
    public Counter() {  
        count++;  // Всички обекти споделят същото count  
    }  
}
```

4.2. Статичен метод

Статичният метод принадлежи на класа, а не на конкретен обект.

```
class MathHelper {  
    public static int Add(int a, int b) {  
        return a + b;  
    }  
}
```

```
// Извикване без създаване на обект:  
int result = MathHelper.Add(5, 3);
```

Разлика между статичен и обикновен метод:

- **Статичен метод:** Принадлежи на класа, извиква се без обект
- **Обикновен метод:** Принадлежи на конкретен обект, извиква се чрез обект

5. Абстрактни класове

5.1. Какво е абстрактен клас?

Абстрактен клас не може да бъде инстанциран. Трябва да се наследява.

```
abstract class Animal {  
    protected string name;  
  
    public Animal(string name) {  
        this.name = name;  
    }  
  
    public abstract void MakeSound(); // Абстрактен метод  
}  
  
class Dog : Animal {  
    public Dog(string name) : base(name) {}  
  
    public override void MakeSound() {  
        Console.WriteLine(name + " лае");  
    }  
}
```


Разлика между абстрактен клас и обикновен клас:

- **Абстрактен клас:** Не можем да създаваме обекти от него
- **Обикновен клас:** Можем да създаваме обекти

6. Интерфейси

6.1. Какво е интерфейс?

Интерфейсът дефинира договор - какво трябва да имплементира класът, но не как.

```
interface IReadable {  
    void Read();  
}  
  
class Book : IReadable {  
    public void Read() {  
        Console.WriteLine("Четене на книга");  
    }  
}
```

Разлика между интерфейс и абстрактен клас:

- **Интерфейс:** Клас може да имплементира множество интерфейси
- **Абстрактен клас:** Клас може да наследява само един абстрактен клас
- **Интерфейс:** Съдържа само декларации на методи
- **Абстрактен клас:** Може да съдържа и имплементация

7. Virtual и Override

7.1. Virtual методи

virtual се използва в базовия клас за да позволи презаписване на метода.

```
class Animal {  
    public virtual void MakeSound() {  
        Console.WriteLine("Животното издава звук");  
    }  
}
```

7.2. Override методи

override се използва в производния клас за да презапише virtual метода.

```
class Dog : Animal {  
    public override void MakeSound() {  
        Console.WriteLine("Кучето лае");  
    }  
}
```

Как работи полиморфизмът:

```
Animal animal = new Dog("Шапо");  
animal.MakeSound(); // Извиква се метода на Dog, не на Animal
```

8. Generics (Шаблонни класове)

8.1. Какво са Generics?

Generics позволяват създаване на класове, които могат да работят с различни типове данни.

```
class Box<T> {  
    private T item;  
  
    public void SetItem(T item) {  
        this.item = item;  
    }  
  
    public T GetItem() {  
        return item;  
    }  
}  
  
// Използване:  
Box<int> intBox = new Box<int>();  
Box<string> stringBox = new Box<string>();
```

9. Обработка на изключения (Exception Handling)

9.1. Try-Catch-Finally блокове

В C# използваме `try-catch-finally` блоковете за обработка на изключения (грешки).

Try блок:

Съдържа кодът, който може да хвърли изключение.

Catch блок:

Улавя и обработва изключенията, които са хвърлени в try блока.

Finally блок:

Винаги се изпълнява, независимо дали е възникнало изключение или не. Използва се за почистване на ресурси.

```
try {
    // Код, който може да хвърли изключение
    int result = 10 / 0; // Това ще хвърли DivideByZeroException
}
catch (DivideByZeroException ex) {
    // Обработка на конкретно изключение
    Console.WriteLine("Грешка: Деление на нула!");
    Console.WriteLine(ex.Message);
}
catch (Exception ex) {
    // Обработка на всички други изключения
    Console.WriteLine("Възникна грешка: " + ex.Message);
}
finally {
    // Този код винаги се изпълнява
    Console.WriteLine("Почистване на ресурси...");
}
```

9.2. Кога се използва всеки блок?

- **try:** Винаги се използва, когато има код, който може да хвърли изключение
- **catch:** Използва се за улавяне и обработка на изключенията
- **finally:** Използва се за код, който трябва да се изпълни винаги (например затваряне на файлове, освобождаване на ресурси)

```
try {
    // Отваряне на файл
    FileStream file = File.OpenRead("data.txt");
    // Работа с файла
}
catch (FileNotFoundException ex) {
    Console.WriteLine("Файлът не е намерен!");
}
finally {
    // Винаги затваряме файла, дори ако е възникнало изключение
}
```

```
// file.Close(); - ако file е дефиниран извън try блока  
}
```

Важни бележки за обработка на изключения:

- Можем да имаме множество `catch` блокове за различни типове изключения
- По-специфичните изключения трябва да са преди по-общите
- `finally` блокът е опционален, но полезен за почистване
- Без `catch` блок, но с `finally` - кодът в `finally` все пак ще се изпълни



Ключови точки за контролната работа

- Разбирате основните принципи на ООП: инкапсулация, наследяване, полиморфизъм, абстракция
- Знаете компонентите на клас: полета, свойства, методи, конструктори
- Разбирате модификаторите за достъп: `private`, `protected`, `public`
- Знаете разликата между статични и обикновени членове
- Разбирате абстрактните класове и интерфейсите
- Знаете как се използват `virtual` и `override` методи
- Разбирате концепцията за `generics`
- Знаете как се обработват изключения чрез `try-catch-finally`
- Разбирате използването на `base` при конструктори
- Можете да допълвате код с правилни ключови думи (`abstract`, `virtual`, `override`, `base`)

Успех на контролната работа! 🎓